

Signals ([Previous Notes](#))

Preliminaries Start

- Definition: A signal is a software interrupt.
- It is sent to your program from the operating system to tell it about some unexpected event.
- For most signals, if your program is not set up to deal with them your program is aborted. Definitely a nice way to make sure that you must deal with them.
- They have similarities to exceptions, but have differences. For example, a signal can be thrown to tell your program about an asynchronous event. Like not being able to deliver a message over an internet connection. If you don't do something about this, your program will terminate.

Important signals (See the Stevens text for a much bigger list):

The following are some of the more important signals

SIGALRM - indicates that a user set alarm has gone off. What do you think this is?

SIGCHLD - indicates that a child process has died. (This signal is ignored by default. This an the exception to the usual processing.) We will have a use for this. Why could this be important? Why can it cause problems?

SIGFPE - indicates that a floating point exception has occurred. Not that important anymore unless the OS is told to throw the exception. How do you find out how to turn on this signal? (**feenableexcept** for Linux) Why would I want to turn them on?

SIGINT - indicates that a user sent interrupt has occurred (E.g. ^c) ^c is the traditional way of terminating a program run from the command line.

SIGKILL - this is a signal that will kill your process. It can't be caught or ignored. We need something like this or a program ignore all signals and keep running regardless of what anyone does except pulling the plug on the computer. It's number is 9.

SIGPIPE - occurs when a write to or a read from a pipe has failed. We will

talk about what this means in class.

SIGTERM - default signal sent by kill command/system call.

SIGUSR1, SIGUSR2 - user defined signals. Nice for simple inter-process communications.

SIGSEGV - segment violation.

Kill Command

Sends a signal to a process. See man page for more details. The following are some of the formats for this command.

kill [-signal] pid

The signal may be a signal number or then name of the signal without SIG in front.

Examples: kill -USR1 231 and kill -9 %2

kill -l (list all signals) This is important in that various UNIX systems have different signals.

Note: this command provides a good tool for testing the effects of signals on processes.

To test, later, I will present the ^z (stops a process), fg (startes a process as a foreground process), and bg (commands. We will use the code in the example: sig1 Now about the **jobs** command lists those processes that are stopped or running in the background.

System calls for signals

The following are some system calls that apply to signals. I have left out the include files that you will need. You can find these in the man/help pages, the text, and my example programs. (Note: a needed include file is signal.h)

int kill(pid_t pid, int sig);

pid - process ID of the process to which the signal is sent.

sig - the signal number that will be sent.

Returns - 0 if successful and -1 if error. There is an error if don't have permission to send signal to a process.

So, with this call, you can terminate processes that you have launched.

int raise(int sig);

Send a signal to the process that calls this function. **Why?????? In truth, not really needed.**

unsigned int alarm(unsigned in seconds);

seconds - the number of seconds before the alarm signal (SIGALRM) will be sent.

returns previous time on alarm.

Notes: If argument zero, cancels alarm. **You can only set up for one alarm at a time.**

int pause();

This is a slow system call. That is, it only returns after the program has been interrupted by a signal and that signal has been handled. Return value is -1; errno is EINTR. This is what all slow call will do when a signal is received and your program has returned from the signal handler. Purpose is clearly to wait for signals to occur.

void (*signal(int signo, void (*func) (int))) (int); Start here 10/8/2025

This function is used to install a signal handler (i.e. a function that will be called if a certain signal occurs) or to describe the actions to be taken when a signal occurs. This function has been supplanted by more recent calls, but it is a good starting point. **We will talk about the more advanced calls later.**

Note: some specifications of this call looks simpler due to the use of typedefs.

signo - the number of the signal.

func - the signal handler. The following is some information about this argument.

The function accepts one argument. This is the number of the signal that caused it to be called. A return from this function brings us back to the instruction that was interrupted or almost. See later discussion. Note: if you use the name of a function in a program, it's address is recorded.

SIG_IGN may be used in the place of func. It means ignore the signal. Basically the signal is thrown away.

SIG_DFL may be used in the place of func to indicate that signals will be handled in the default manner.

Returns pointer to previous disposition. That is the previous handler. This is handy to have in that we can later reinstall the original signal handler. (Most of the time you won't need this information.) SIG_ERR is returned if there is an error.

Note: there is some fancy casting to get the constants arguments (E.g. SIG_IGN) to be accepted as an argument. Luckily we don't have to do it.

Further notes on signal processing

- Slow system calls - system calls that can block for an indefinite amount of time. When a signal handler returns, these functions return indicating an error. `errno = EINTR` in these cases. [We can work around this, if we don't want it to occur using more recent system calls.](#)
- The pause and read (from pipe or keyboard) system calls are slow system calls. We will have more.
- Signals are not queued. If the same signal is sent to your process a hundred times, it does not necessarily mean that the signal handler will be called a hundred times. This can be the source of numerous bugs. Why do you think that UNIX does this?
- Writing signal handlers: Keep simple. Problem with non-reentrant function in handler. See text for system calls that are safe (reentrant.) We will discuss further.
- If your program has a main loop, your signal handler should set a global variable to indicate that a signal should be handled. Why?
- For some older versions of UNIX, you must reinstall the signal handler after it has been called. [Linux does not require a reinstall. You might see the reinstall in older code or from more vintage programmers, but it no longer needed. I have the reinstall commented out in my examples.](#)
- Mention volatile variables with respect to optimization. Not using volatile variables can cause cases where a debug version of the code will work and a release version will fail.

Some of the issues discussed here are remedied by the newer calls.

See examples. [sig1.c](#), [sig2.c](#), [sig3.c](#), [sig4.c](#), [sig5.c](#), [sig6.c](#), and [sig7.c](#) I used C for these examples as a way of making sure that you understand not only C++ but also C code. We will let the C++ compiler run these. Otherwise we have to be careful about some of the C++ includes.

Notes:

1. I use sig1.c to demonstrate using ^c to terminate a program. (Note: since the current directory is not in the PATH, it will not run if you type "sig1" in the directory where the executable resides. Type "./sig1" instead.) I will also use ^z to stop it. The **jobs** command to look at programs that we are playing with. I will also show the use of the fg (foreground) and bg (background) commands. I will also use the kill command to send a signal. I will send a floating point exception, SIGUSR1 and SIGKILL. Note that a signal is delivered to a stopped program when it is restarted. (Note how you can run a process in the background using "&".)
2. The program sig2.c will demonstrate how we can ignore signals. A nice thing to be able to do. How can we ignore all signals? Well, all but one.
3. In sig3.c I install signal handlers for SIGUSR1 and SIGUSR2. Note we no longer have to reinstall the signal handler. Note also how we return from a slow system call. (sleep) Strange in that we are not catching the signals.
4. The sig4.c example demonstrates the alarm function and the use of a SIGALRM handler. Note that the pause call gets interrupted when the alarm returns. If this was a sleep calls, what is the return value of sleep if this occurs. Let's try it.
5. sig5.c shows how difficult it is to cause a floating point exception. (A strange change has occurred in Linux and VS where the floating point exceptions are turned on for debug but not for release.. For other versions, it is good to check. BTW, csa might still not throw the exception. Microsoft Copilot claims that Linux still masks floating point exceptions.) The following seems to be obsolete for Linux: This is due the IEEE floating point specification. I did force it in a weird way. Note what happens when you return from the exception. This shows that some exceptions should not be returned from. PS You should know how to turn on floating point exceptions or at least how to find out how to turn them on.
6. sig6.c forces a floating point exception. This is useful in testing. I could have used raise as well. There is a small risk of the code Lab7 will give you a chance to do better.
7. sig7.c shows how processing continues from where it was interrupted after a signal handler is called and returns. (We saw this in sig5 example.) Note the volatile keyword. Do we need it? Sort of...

Lab6

Write a program will compute prime numbers and write them to a file that will be specified as a command-line argument. The program will display the most recent prime number generated if it receives a SIGUSR1 signal. Also, there will be additional command-line arguments which will specify **the times in seconds from the beginning of the run** at which the current prime will be displayed. Note: we are allowing multiple

times to be in the command-line. **This latter is trickier than it might seem.**

Lab 7

Find a way to turn on floating point exceptions and write a small program to test this by doing floating point arithmetic that will cause an exception. So, you must find a way to turn on floating point exceptions. **For the more recent version of Linux, see if you can find a way to turn them on. This is now only an issue when you compile for release.**

Replacements for the Signal System Call

Beside the importance of the material discussed in this section, it is important in and of itself in that we can see the evolution of one aspect of an operating system.

The **signal** system call is a bit antiquated. (It still is useful.) Over the years, people have decided that we need a lot more flexibility in handling signals. **For example, the ability to temporarily block signals is quite useful.** (Blocking is different than ignoring.) There are a number of newer system calls to deal with signals. These have been around for a while.

Since some of these newer calls require that you specify sets of signals, we will discuss how to do this first.

Signal Sets

Signal sets - the `sigset_t` data type is used to represent a set of signals. A bit map would be nice if it could be a `int`, but this would require that there be less than or equal to 32 signals. Linux currently has 64 signals. So, a long (64 bit integer) would currently work. On some Unix systems `sigset_t` is an array. How would we find out how this data type is represented? Should we care? **NO! This is because the representation of sets of signals can change.**

The following are calls to manipulate signal sets:

```
int sigemptyset( sigset_t *set);    // Creates an empty set.

int sigfillset( sigset_t *set);      // creates a set where every signal is set.

int sigaddset(sigset_t *set, int signo); // Adds a signal to a set.

int sigdelset(sigset_t *set, int signo); // Deletes a signal from a set.
```

For all of these calls, returns 0 if OK and -1 if error. You might wonder how you can get an error on some of these. We will talk about it.

```
int sigismember (constant sigset_t *set, int signo); // Tests for membership in
the set.
```

returns 1 if true and 0 if false.

The names indicates what these function do. We will discuss in class.

sigprocmask

The signal mask is kept by OS for each process as a description of which signals are currently blocked.

```
int sigprocmask( int how, const sigset_t *set, sigset_t *oset );
```

how- indicates what we are trying to accomplish. The values for how are:

SIG_BLOCK - block all signals in the **set** argument.

SIG_UNBLOCK - unblock all signals in the **set** argument.

SIG_SETMASK. - make the **set** argument the new signal mask.

set - a signal set which will effect the signal mask in the manner described by the how argument. NULL means don't use.

oset - returns the previous signal mask. NULL for this argument means don't want/need this information.

Returns 0 if OK and -1 if error.

The following function can be used to find out what signals are blocked.

sigpending

```
int sigpending( sigset_t *set );
```

set - returns a the set of pending signals. That is blocked signals.

Return 0 if successful and -1 if fails.

Can use the previously discussed sigismember to determine if a given signal is pending. It determines if the signal is a member of a set. It is useful when you have signals blocked and you want to know what signals are waiting. Reminder: the prototype for sigismember is:

```
int sigismember(sigset_t *set, int signo);
```

sigaction

The following call is a replacement for the signal system call. For many UNIX programmers it supersedes the **signal** system call. In other words, it is a good idea to use this call most of the time. I tend to use the **signal** call to set a handler for all signals and other relatively simple tasks.

```
int sigaction( int signo, const struct sigaction *act, struct sigaction *oact );
```

signo - is the signal number.

act - is the action that we want to happen when the signal occurs. Make it NULL if we are not setting an action.

oact - is the previous action. Make it NULL if we don't care. The format of the sigaction structure is as follows:

```
struct sigaction {  
  
    void (*sa_handler) ( int signo ); /* Address of signal handler. Or  
    SIG_IGN or SIG_DFL */  
  
    sigset_t sa_mask; /* Additional signals to block when in the handler.  
    */  
  
    int sa_flags; /* Options. */  
  
};
```

options: SA_RESTART (restart slow system calls automatically - **this can be very helpful**), SA_NOCLDWAIT (if signo = SIGCHLD, remove zombies), others to emulate old style signal call. As always, there are a lot more options. Most of these are pretty obscure.

Note: **when the signal handler is executed, the signal that is being processed is automatically blocked**. This is very nice! You can disable this feature with an option flag. Also, you don't have to reinstall the signal handler. This is no longer an issue with the signal system call on Linux. If you use other versions of UNIX, verify this.

sigsetjmp and siglongjmp Calls

- sigsetjmp and siglongjmp systems calls.
- They replace the older setjmp and longjmp which we won't talk about. (This is no need to talk about these calls in that the calls presented here are a superset of the these two.)
- These calls are used as giant goto's. (The calls, in addition to this goto functionality, can also be used to save and restore the signal mask.)

- The go to statement is usually considered evil. But, these calls do have a reason to exist but you should not be using them often. Among other uses, they allow us to return from a signal handler to a specified place.
- In class I will talk about the importance of this call in making sure a cleanup occurs and a destructor is called.

```
#include <setjmp.h>
```

```
int sigsetjmp( sigjmp_buf env, int savemask );
```

env - used to record the program's environment. (This includes the stack) It was an array the last time I looked. That is, sigjmp_buf was typedef'd for an array. We don't really care what it is, in that the type can change.

savemask - indicates whether or not to save the signal mask. != 0 means save mask..

Returns == 0 if called directly and !=0 if coming back from a siglongjmp. (See its description.)

```
int siglongjmp( sigjmp_buf env, int val );
```

env - the program environment recorded by sigsetjmp. **So the environment recorded in sigsetjmp must be "seeable" by this call. How do we do that?**

val - return value for sigsetjmp. As you can guess, value had better not be zero.

Returns -1 if it fails. If it succeeds, it will not return.

sigsuspend

The following call defends against race conditions by doing two things at once. It is a combination of setting the signal mask and a pause statement. When a return is made from a signal handler, execution will continue.

```
#include <signal.h>
```

```
int sigsuspend( const sigset_t *set );
```

Sets the signal mask and does the equivalent of a pause in one step.

set - value for signal mask.

returns -1 with `errno = EINTR` when a signal occurs.

Used to avoid the vulnerability of doing this with a `sigprocmask` and `pause`. Why is using the two calls a risk?

sleep

`unsigned int sleep( unsigned int seconds );`

Sleep system call. Lets a process suspend for a specified number of seconds. Returns the unused sleep time. We can get unused sleep time due to a return from a signal handler. There are other sleep calls in Linux where smaller time intervals are permitted. If you need one, do a search. (For example, there is a `nanosleep` function.

The following are some examples on using these calls. [sig8.c](#), [sig9.c](#) and [sig10.c](#)

1. `sig8.c` demonstrates how we can block signals. If you run this in the foreground, life will be interesting. We will do it. Note: it even blocks the effect of a `^z`. We can kill it with a 9 signal. (SIGKILL)
2. `sig9.c` shows the use of `sigaction` to install a signal handler for two signals.
3. `sig10.c` demonstrates the `longjmp` call. Note the need to use a global variable to hold the environment. Note volatile issue which does not show up when you compile for debug. We will discuss how to use it to force a destructor to be called.
What could happen if compiled for release and the variable `duck` is not volatile? Let's talk about why this is useful.